

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..23

MACOS SETUP

 Copy as Markdown

Set up authentication on macOS

Serverpod stores sign-in tokens in the macOS Keychain so sessions survive app restarts. Sandboxed macOS apps need a Keychain Sharing entitlement before they can read or write that data. This guide adds the entitlement to your Flutter project. It takes a few minutes and applies to every sign-in method (email, Google, Apple, and others).

Before you start

- A Serverpod project with the [authentication module](#) enabled (the default when you create a project with `serverpod create`).
- A Flutter app that calls `client.auth.initialize()` on startup (included in the project template).
- Xcode installed, if you prefer adding the entitlement through the Xcode UI.

Add the Keychain Sharing entitlement

Add the following to both entitlements files in your Flutter app:

- `macos/Runner/DebugProfile.entitlements`
- `macos/Runner/Release.entitlements`

```
macos/Runner/DebugProfile.entitlements
```

```
<key>keychain-access-groups</key>
<array/>
```

Copy the same block into `macos/Runner/Release.entitlements`.

The empty array enables Keychain access without sharing keychain items with other apps. Leave empty unless you have a specific reason to add a group identifier.



Ask AI

Rebuild and run your app:

```
cd <project>_flutter
flutter run -d macos
```

Verify

1. Sign in through any configured provider (email is the quickest to test).
2. Confirm the app shows your signed-in content without an error.
3. Stop and restart the app. The user should still be signed in.

If sign-in still fails, see [Troubleshooting](#).

Advanced

Add the entitlement in Xcode

1. Open `macos/Runner.xcworkspace` in Xcode.
2. Select the **Runner** target, then open **Signing & Capabilities**.
3. Click + **Capability** and add **Keychain Sharing**.
4. Repeat for **Debug**, **Profile**, and **Release** build configurations if they use separate entitlements files.

Xcode writes the same Keychain Sharing entry (`keychain-access-groups`) into your entitlements files.

Use the login keychain for local development

If you run unsigned macOS builds locally and want to avoid Keychain Sharing entitlements entirely, configure secure storage to use the login keychain instead of the Data Protection Keychain. Override the session manager storage where `initializeClient()` creates the client in your app's `lib/client.dart`:

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

client = Client(await serverUrl)
  ..connectivityMonitor = FlutterConnectivityMonitor()
  ..authSessionManager = FlutterAuthSessionManager(
    storage: SecureClientAuthSuccessStorage(
      secureStorage: const FlutterSecureStorage(
        mOptions: MacOSOptions(
          usesDataProtectionKeychain: false,
        ),
      ),
    ),
  );
```

This path requires `flutter_secure_storage` 10.3.0 or later, where `usesDataProtectionKeychain: false` started taking effect. Projects created with `serverpod create` already resolve a compatible version. If your project was created earlier, run `flutter pub upgrade flutter_secure_storage` to update the lockfile.

The login keychain works for local development. For production macOS builds, prefer the Keychain Sharing entitlement in [Add the Keychain Sharing entitlement](#) above.

WARNING

If you add a non-empty value to `keychain-access-groups` (for example, `$(AppIdentifierPrefix)com.example.app`), Xcode may require an Apple Developer account and a provisioning profile to sign the app.

Troubleshooting

Sign-in fails with "A required entitlement isn't present"

Problem: After entering valid credentials, sign-in fails with an error similar to:

```
PlatformException(-34018, A required entitlement isn't present, ...)
```

The server accepts the login, but the Flutter app cannot save the session.

Cause: The macOS app is missing the Keychain Sharing entitlement (`keychain-access-groups`). When you create a project with `serverpod create`, the CLI adds network entitlements automatically, but not Keychain access.

Resolution: Follow [Add the Keychain Sharing entitlement](#) above. Rebuild the app after editing the entitlements files.

Session is lost after restarting the app

Problem: Sign-in succeeds, but the user is signed out after closing and reopening the app.

Cause: The app cannot read the stored session from the Keychain, usually because the entitlement is missing or the app was not rebuilt after adding it.

Resolution: Confirm both entitlements files contain the Keychain Sharing entry (`keychain-access-groups`), rebuild with `flutter run -d macos`, then sign in again and restart the app.

Related

- [Get started with authentication](#): turn on the sign-in screen and test your first login
- [Authentication basics](#): check sign-in state and listen for auth changes on the client
- [flutter_secure_storage macOS setup](#): platform details for secure storage on macOS